## ✅ Unit I: Fundamentals

**1. Define an Embedded System and distinguish it from a General Purpose PC.**
An embedded system is a **special-purpose computer** designed to perform a **specific task**.
Example: washing machine, microwave.

- Embedded System → single task, real-time, limited memory
- General Purpose PC → multiple tasks, flexible, high memory

---

**2. Draw and explain the basic functional block diagram of an embedded system.**
Main blocks:

- **Input (Sensors)** → collects data
- **Processor (Microcontroller)** → processes data
- **Memory** → stores program/data
- **Output (Actuators)** → gives result
- **Power Supply** → provides energy

---

**3. List and explain four "Operational Quality Attributes" of an embedded system.**

- **Reliability** → works continuously without failure
- **Efficiency** → uses less power and resources
- **Maintainability** → easy to repair/update
- **Safety** → avoids damage or hazards

---

**4. What is the significance of "Determinism" in embedded system behavior?**
Determinism means **predictable output at fixed time**.
Important for real-time systems like airbags and medical devices.

---

**5. Classify embedded systems based on generation and complexity.**

- **Small Scale** → simple, 8-bit controller
- **Medium Scale** → 16/32-bit controller
- **Complex Systems** → RTOS-based, high processing

---

## ✅ Unit II: Processor Architecture

**1. Compare and contrast RISC and CISC architectures with examples.**

- **RISC** → simple instructions, fast (Example: ARM)
- **CISC** → complex instructions, slower (Example: Intel x86)

---

**2. Explain the advantages of Harvard architecture in high-speed embedded processing.**
Uses separate memory for data and instructions → allows **parallel access** → faster execution.

---

**3. Define System-on-Chip (SoC) and explain its benefits over traditional discrete designs.**
SoC is a **single chip containing CPU, memory, and peripherals**.
Benefits: small size, low power, low cost, high speed

---

**4. Describe the basic internal architecture of an ARM or AVR microcontroller.**
Includes:

- CPU
- Registers
- ALU
- Memory interface
- I/O ports
- Timers & interrupts

---

**5. Explain the different types of memory (SRAM, DRAM, NVRAM) used in embedded designs.**

- **SRAM** → fast, volatile
- **DRAM** → slower, needs refresh
- **NVRAM** → non-volatile, retains data

---

## ✅ Unit III: Programming & Development

**1. Why is "Embedded C" preferred over high-level C for firmware development?**
Because it provides **direct hardware control**, better memory handling, and supports registers/interrupts.

---

**2. Explain the role of a Cross-Compiler in the embedded development life cycle.**
A cross-compiler converts code written on a PC into machine code for the **target microcontroller**.

---

**3. Describe the steps involved in transforming a source program into an executable image (Flashing).**

1. Write program
2. Compile
3. Generate HEX file
4. Upload to microcontroller

---

**4. Compare the features of the Keil IDE and Arduino IDE for professional vs. hobbyist use.**

- **Keil IDE** → professional, detailed control
- **Arduino IDE** → beginner-friendly, simple

---

**5. What is "In-Circuit Emulation" (ICE) and how does it assist in hardware debugging?**
ICE allows testing and debugging the system **while it is running**, helping detect real-time errors.

---

## ✅ Unit IV: RTOS Concepts

**1. Distinguish between Hard Real-Time and Soft Real-Time systems with examples.**

- **Hard RT** → strict deadline (airbag system)
- **Soft RT** → delay acceptable (video streaming)

---

**2. Explain the "Preemptive Scheduling" mechanism used in RTOS kernels.**
Higher priority task **interrupts and replaces** lower priority task execution.

---

**3. Define Interrupt Latency and explain its impact on system performance.**
Time between interrupt signal and response.
Lower latency → faster system performance

---

**4. What is a "Task Control Block" (TCB) and what information does it store?**
TCB stores:

- Task state
- Priority
- Register values
- Stack information

---

**5. Describe the role of Semaphores and Mutexes in task synchronization.**

- **Semaphore** → signaling between tasks
- **Mutex** → protects shared resources

---

✅ **Unit V: Emerging Trends**

**1. Explain the architecture of an IoT-enabled embedded device.**
Sensors → Microcontroller → Communication (WiFi/Bluetooth) → Cloud

---

**2. What are the challenges of implementing Machine Learning models on resource-constrained embedded hardware?**

- Limited memory
- Low processing power
- Power consumption issues

---

**3. Discuss the security challenges specific to wearable medical devices.**

- Data privacy issues
- Unauthorized access
- Weak security mechanisms

---

**4. How is "Edge Computing" changing the way embedded systems handle data?**
Data is processed locally → faster response, less cloud dependency

---

**5. Explain the role of embedded systems in the development of autonomous vehicles.**
Used for sensor processing, decision making, and vehicle control

---

## ✅ Unit VI: Case Studies

**1. Compare Raspberry Pi and Arduino in terms of OS capability and hardware control.**

- Raspberry Pi → supports OS, complex tasks
- Arduino → no OS, simple control tasks

---

**2. Design a high-level block diagram for a Smart Home Automation system using ESP32.**
Sensors → ESP32 → WiFi → Mobile App → Control devices

---

**3. Analyze the role of embedded controllers in an Automotive Airbag Deployment system.**
Sensors detect collision → controller processes → airbag deploys instantly

---

**4. What makes the ESP32 a popular choice for low-power IoT applications?**

- Built-in WiFi & Bluetooth
- Low power consumption
- Cost-effective
- High performance

---

**5. Discuss a case study of an embedded system used in environmental monitoring.**
Sensors collect environmental data → microcontroller processes → data sent to cloud for monitoring